

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Weightage:	30% of CIA
Student Name:	G Hari Prasad	Reg. No.:	192525273
Section / Batch:	B	Date:	24-08-2026

Q35. Graph Bandwidth Minimization (NP-Complete Problem)

Arrange graph vertices linearly such that the maximum edge length is minimized. Clearly define constraints related to vertex ordering and edge distance. Analyze how constraints impact feasible arrangements. Develop a backtracking-based solution. Apply pruning to reduce search space. Justify correctness and discuss NP-Complete complexity.

1. Problem Understanding

Given an undirected graph $G = (V, E)$ with $n = |V|$ vertices, a **linear arrangement** is a bijection $f : V \rightarrow \{0, 1, 2, \dots, n-1\}$ that assigns every vertex a unique position on a number line. For every edge $(u, v) \in E$, its **edge length** is defined as $|f(u) - f(v)|$. The **bandwidth** of the arrangement f is the maximum edge length over all edges: $B(f) = \max_{(u,v) \in E} |f(u) - f(v)|$. The **Graph Bandwidth Minimization Problem** asks for the arrangement f^* that minimizes $B(f)$ over all possible permutations of vertices.

This is naturally a **combinatorial optimization** problem: the search space is the set of all $n!$ permutations of vertices, and we must find the permutation that satisfies the objective (minimum maximum edge stretch) while respecting the structural constraints imposed by the edge set E .

2. Constraints Identified

Constraint	Description
Bijection Constraint	Every vertex must be assigned exactly one distinct position in $\{0, \dots, n-1\}$; no two vertices can share a position (permutation constraint).
Completeness Constraint	All n vertices must be placed before an arrangement is considered valid/complete.
Edge-Length Constraint	For every edge (u,v) , the value $ \text{pos}(u) - \text{pos}(v) $ contributes to the bandwidth; the arrangement must keep this as small as possible for all edges simultaneously.
Bandwidth (Objective) Constraint	The final answer must minimize the maximum edge length, i.e. minimize $\max \text{pos}(u) - \text{pos}(v) $ over all edges - a min-max constraint.
Ordering Dependency	The feasibility of placing a vertex at a given position depends on the positions already assigned to its neighbours (local constraint propagation).

3. Impact of Constraints on Feasible Arrangements

The bijection and completeness constraints shrink the raw search space to exactly $n!$ candidate arrangements, but the min-max bandwidth objective is what makes the problem hard: unlike a sum-based objective, a single badly-placed edge can dominate the entire arrangement's cost, so local decisions cannot be evaluated independently – every partial placement must be checked against all already-placed neighbours. High-degree vertices are the most constrained: their neighbours must all be kept close in position, which sharply reduces the number of feasible continuations from any given partial arrangement. This observation is exactly what enables an effective pruning strategy in backtracking: a partial arrangement whose already-realized edge length equals or exceeds the best bandwidth found so far can never lead to a better solution and is discarded early.

4. State-Space Representation

The state-space tree has depth n . Each level i (0-indexed) represents the decision "which unused vertex occupies position i ". A root-to-leaf path of length n therefore represents one complete permutation (linear arrangement). At every node, the branching factor is the number of vertices not yet placed, so the raw tree has $n!$ leaves. Each internal node also carries a **partial bandwidth value** – the maximum edge length among edges whose both endpoints have already been assigned positions on that path – which is used as the bounding function for pruning.

5. Backtracking Algorithm with Pruning

Pseudocode:

```
ALGORITHM BandwidthBacktrack(G, n)
  best_bw <- INFINITY
  best_order <- NULL
  position[0..n-1] <- -1          // position of each vertex
  used[0..n-1] <- FALSE

  PROCEDURE Backtrack(pos):
    IF pos == n:                  // complete arrangement
      bw <- MaxEdgeLength(position)
      IF bw < best_bw:
        best_bw <- bw
        best_order <- COPY(order)
      RETURN

  FOR each vertex v with used[v] = FALSE:
    position[v] <- pos
    used[v] <- TRUE

    partial_bw <- MaxEdgeLength(position) // only realized edges
    IF partial_bw < best_bw:              // PRUNING STEP
      Backtrack(pos + 1)
    // else: branch discarded, bound already violated

    used[v] <- FALSE                    // undo / backtrack
    position[v] <- -1

  Backtrack(0)
  RETURN best_bw, best_order
```

Pruning rule used: Branch-and-bound pruning on the objective value. Whenever placing a vertex creates a realized edge length $\geq$ the best bandwidth found so far, the entire subtree rooted at that partial arrangement is abandoned, since completing it can never produce a smaller bandwidth than the current best. This is the classic backtracking constraint-check ("is the partial solution still promising?") applied to a min-max objective.

6. Python Implementation

```
"""
Graph Bandwidth Minimization using Backtracking with Pruning
Q35 - Constraint Based Problem Solving
"""

def read_graph():
    # Example graph (adjacency list)
    edges = [(0,1), (0,2), (1,2), (1,3), (2,4), (3,4)]
    n = 5
    return n, edges

def bandwidth_backtracking(n, edges):
    best = {"bw": float('inf'), "order": None}
    position = [-1] * n      # position[v] = index in linear arrangement
    used = [False] * n
    order = [-1] * n

    def current_max_bandwidth_partial():
        mx = 0
        for (u, v) in edges:
            if position[u] != -1 and position[v] != -1:
                mx = max(mx, abs(position[u] - position[v]))
        return mx

    def backtrack(pos):
        if pos == n:
            bw = current_max_bandwidth_partial()
            if bw < best["bw"]:
                best["bw"] = bw
                best["order"] = order.copy()
            return
        for v in range(n):
            if used[v]:
                continue
            position[v] = pos
            order[pos] = v
            used[v] = True

            # PRUNING: if partial bandwidth already >= best known,
            # discard this branch (bound cannot improve)
            partial_bw = current_max_bandwidth_partial()
            if partial_bw < best["bw"]:
                backtrack(pos + 1)

            used[v] = False
            position[v] = -1
            order[pos] = -1

    backtrack(0)
    return best["bw"], best["order"]

if __name__ == "__main__":
    n, edges = read_graph()
    print("Graph vertices:", list(range(n)))
    print("Graph edges   :", edges)
    print("-" * 50)

    bw, order = bandwidth_backtracking(n, edges)
    print(f"Minimum Bandwidth found      : {bw}")
    print(f"Optimal linear arrangement   : {order}")

    print("-" * 50)
    print("Edge lengths under optimal arrangement:")
    pos = {v: i for i, v in enumerate(order)}
    for (u, v) in edges:
        print(f"   edge ({u},{v}) -> |{pos[u]} - {pos[v]}| = {abs(pos[u]-pos[v])}")
```

7. Program Output

Executed on the sample graph $V = \{0,1,2,3,4\}$, $E = \{(0,1),(0,2),(1,2),(1,3),(2,4),(3,4)\}$:

```
Graph vertices: [0, 1, 2, 3, 4]
Graph edges   : [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
-----
Minimum Bandwidth found      : 2
Optimal linear arrangement   : [0, 1, 2, 3, 4]
-----
Edge lengths under optimal arrangement:
edge (0,1) -> |0 - 1| = 1
edge (0,2) -> |0 - 2| = 2
edge (1,2) -> |1 - 2| = 1
edge (1,3) -> |1 - 3| = 2
edge (2,4) -> |2 - 4| = 2
edge (3,4) -> |3 - 4| = 1
```

Interpretation: The algorithm explores permutations with branch-and-bound pruning and returns the identity ordering [0,1,2,3,4] as optimal, giving a minimum bandwidth of 2 (the edges (0,2) and (1,3) are the bottleneck edges, each stretching 2 positions).

8. Correctness Justification

Completeness: The algorithm systematically explores every position assignment via full recursive branching (for each free position, every unused vertex is tried), so in the absence of pruning it would enumerate all $n!$ permutations and is therefore guaranteed to reach the true optimum. **Soundness of pruning:** the bound used, "discard if partial bandwidth $\geq$ best found so far," is monotonic – adding more vertices to a partial arrangement can only keep the realized bandwidth the same or increase it, never decrease it (since `MaxEdgeLength` is computed only over realized edges and is non-decreasing as more edges are realized). Hence any branch pruned this way could not have produced a strictly better solution than the current best, so no optimal solution is ever lost. **Termination:** the recursion depth is bounded by n and each level has a finite branching factor, so the algorithm always terminates and returns the true minimum bandwidth and a corresponding optimal arrangement.

9. Complexity Analysis

Without pruning: the state-space tree has $n!$ leaves, and evaluating a complete arrangement costs $O(|E|)$, giving worst-case time complexity $O(n! \cdot |E|)$. **With branch-and-bound pruning:** the effective number of explored nodes is reduced considerably in practice (especially for sparse or structured graphs) because entire subtrees are cut as soon as a partial bandwidth violates the current best bound; however, the **worst-case** complexity remains exponential, $O(n!)$, since no polynomial bound on the pruned tree size can be guaranteed for adversarial graphs. Space complexity is $O(n)$ for the recursion stack and position/used arrays (excluding the $O(|E|)$ graph storage).

10. Complexity Class: NP-Complete

The **decision version** of the problem is: "Given graph G and integer k , does there exist a linear arrangement of V with bandwidth $\leq k$?" This decision problem is known to be **NP-Complete** (Papadimitriou, 1976), established as follows:

(a) Bandwidth $\in$ NP:

Given a candidate arrangement (certificate) as a permutation of vertices, we can verify in polynomial time, $O(|E|)$, whether every edge length is $\leq k$ by simply computing $|\text{pos}(u) - \text{pos}(v)|$ for each edge and comparing it to k . Hence a "yes" instance can be verified in polynomial time, so the problem belongs to NP.

(b) Bandwidth is NP-Hard:

NP-hardness is shown by a polynomial-time reduction from a known NP-Complete problem (classically the Hamiltonian Path Problem, and in Papadimitriou's original proof from a restricted form of the problem

built on 3-Partition-style gadgets). Intuitively, forcing bandwidth $\leq k$ restricts every pair of adjacent vertices to be within k positions of one another; by constructing gadget graphs whose only bandwidth- k arrangements correspond to valid solutions of the source problem, membership in the source problem is shown to be equivalent to the existence of a bandwidth- k arrangement of the constructed graph. Since the reduction runs in polynomial time and a known NP-Complete problem reduces to Bandwidth, Bandwidth is NP-hard.

Since Bandwidth Minimization is both in NP and NP-hard, it is **NP-Complete**. Consequently, unless $P = NP$, no polynomial-time algorithm is known (or believed to exist) that solves it exactly for all graphs, which is exactly why an exponential backtracking approach (accelerated with branch-and-bound pruning, or in practice with heuristics such as Cuthill–McKee for large graphs) is the standard exact-solution strategy.

11. Conclusion

The Graph Bandwidth Minimization problem was modelled as a constrained permutation search, its governing constraints (bijection, completeness, min-max edge length) were identified and analyzed, and a backtracking algorithm with branch-and-bound pruning was designed, implemented, and verified experimentally. The algorithm is provably correct and complete, runs in exponential worst-case time consistent with the problem's proven **NP-Complete** status, and the pruning strategy substantially reduces the practical search space while preserving optimality.

I **G Hari Prasad (Reg. No. 192525273)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the Student: **G Hari Prasad**

Date: **24-08-2026**